

TokenStream CRA Risk Assessment

Product And Environment Context

Status: Draft Date: 2026-08-26 Product: TokenStream Assessed version: 0.1.0 Assessment type: CRA voluntary readiness risk assessment; no EU Declaration of Conformity or CE marking.

1. Purpose Of This Context

This document defines the product and operating environment before individual threats are identified. It should be read together with the risk assessment methodology and the Threat Dragon STRIDE model.

The context deliberately reflects TokenStream as it exists: open-source, non-commercial, self-hosted software without enterprise support, warranty, insurance, or a guaranteed support service. The risk assessment may still use CRA concepts as a voluntary readiness framework, but it must not imply that TokenStream is CE-marked, placed on the market as a commercial product, or supported under a contractual support-period commitment.

2. Intended Purpose

TokenStream is a policy-aware routing and orchestration runtime for LLMs and LLM-used tools, including retrieval-augmented generation (RAG) and optional MCP-backed tools. It is designed to sit behind client applications and provide a governed runtime boundary between those applications, model providers, retrieval infrastructure, and tool services.

Its intended purpose is to:

- expose OpenAI-compatible model and retrieval-facing APIs for client applications;
- route model requests to configured providers and models;
- enforce pipeline policies for providers, models, tools, corpora, filters, token limits, and retrieval limits;
- expose retrieval as a governed tool or as deterministic structured RAG endpoints;
- ingest source material into corpus-scoped lexical, vector, and graph-aware retrieval indexes;
- manage provider records, policies, corpora, sources, ingestion jobs, machine API keys, users, RAG settings, retrieval profiles, processors, and MCP settings;
- create auditable operational traces between client applications, policies, LLM providers, retrieval results, tools, and administrative configuration changes.

TokenStream is domain-neutral. It does not define the end-user product workflow, final user experience, domain-specific prompt policy, or business logic of the client application that uses it.

3. Reasonably Foreseeable Use

Reasonably foreseeable use is a self-hosted deployment where TokenStream runs behind one or more client application backends. Normal users of the client application do not interact directly with TokenStream.

Expected use includes:

- a client application backend calls `orchestrator-api` for OpenAI-compatible chat completions, model discovery, and structured RAG queries;
- a client application authenticates with a machine API key when service-level authentication is enabled;
- operators configure providers, policies, corpora, sources, machine keys, MCP servers, retrieval settings, and users through the browser admin UI and management API;
- administrators create corpora, register or upload sources, start ingestion jobs, and check corpus readiness before using retrieved context;
- ingestion-worker processes registered or uploaded sources, chunks content, calls embedding services, and writes retrieval artifacts;
- `retrieval-api` is normally called by `orchestrator-api` or trusted internal clients, not by public end-user devices;
- optional Open WebUI may be deployed as a local client of the orchestration API, but it is not the primary TokenStream administration boundary;
- deployments use local secrets from environment variables, runtime snapshots, and scoped machine keys rather than committing provider credentials into source-controlled JSON files.

The admin interface is an operations surface. It should be accessible only to trusted operators and should not be exposed to ordinary application users.

4. Reasonably Foreseeable Misuse

The following misuse cases are reasonably foreseeable and should be considered during threat identification:

- exposing `dev-ui`, `/v1/management/*`, or internal control-plane routes to untrusted users or the public internet;
- using or leaving enabled the development bootstrap administrator account (`admin / admin`) outside a trusted local development environment;
- failing to rotate bootstrap, provider, machine, MinIO, Open WebUI, reload, or internal service tokens after setup, disclosure, logging, or accidental sharing;
- storing real provider API keys, machine keys, passwords, session tokens, private keys, or internal tokens in committed files, provider registries, logs, screenshots, shell history, or chat transcripts;
- giving client applications broad machine keys, `admin:*`, write scopes, unrestricted provider/model access, or unrestricted corpus access when least privilege would be sufficient;

- calling `retrieval-api` directly from normal client applications without treating it as a trusted internal boundary;
- exposing Qdrant, MinIO, the MinIO console, TEI embedder, Open WebUI, or other internal services without network segmentation and authentication appropriate to the deployment;
- enabling optional MCP servers or tools without strict policy allowlists, capability review, network restrictions, and tool-output handling;
- configuring policies that allow unintended providers, models, corpora, retrieval filters, tools, or token limits;
- mixing unrelated tenants, customers, products, or confidentiality domains into the same corpus;
- relying on retrieval results before ingestion has completed or after source material has changed without reingestion;
- ingesting untrusted URLs, archives, uploaded files, or repositories without domain allowlisting, file-size limits, malware scanning, provenance checks, parser hardening, and review of generated metadata;
- treating source-backed answers as complete or safe when citations are missing, stale, contradictory, or low confidence;
- forwarding excessive prompt context, retrieved chunks, sensitive source material, or tool payloads to external LLM providers or MCP services;
- operating without TLS, reverse-proxy hardening, backup/restore procedures, log redaction, monitoring, rate limits, or resource limits where the deployment is reachable beyond a local trusted machine;
- assuming enterprise support, warranty, guaranteed patch delivery, or CRA CE marking exists for this open-source, non-commercial distribution;
- continuing to operate an unsupported or unmaintained release line without self-maintenance, forking, compensating controls, or migration.

5. Users And Actors

Actor	Description	Trust level
Human administrator / operator	Trusted person who signs into `dev-ui` to configure providers, policies, users, keys, corpora, sources, jobs, RAG settings, retrieval profiles, processors, and MCP settings.	High trust; administrative boundary.
TokenStream maintainer	OSS maintainer who may review issues, publish releases, and respond to vulnerability reports on a best-effort basis.	Trusted for upstream code and releases, but not a contractual support provider.
Client application backend	Application-owned backend service that calls `orchestrator-api` using a machine key or deployment-level bearer token.	Trusted according to issued scopes and policy constraints.

Actor	Description	Trust level
Client application end user	End user of an application that indirectly uses TokenStream through the application backend.	Untrusted or partially trusted; should not directly access TokenStream admin or internal APIs.
Service account / machine key subject	Non-human identity used for chat, RAG, corpus, ingestion, or automation workflows.	Trust depends on scope, key handling, and policy restrictions.
TokenStream internal services	`orchestrator-api`, `retrieval-api`, `ingestion-worker`, `dev-ui`/`config_auth`, shared packages, and runtime snapshot loaders.	Trusted service components within the deployment boundary.
Data services	Qdrant, MinIO-compatible object storage, TEI embedder, SQLite lexical/graph stores, and local filesystem volumes.	Trusted infrastructure components when network-restricted and correctly configured.
External model providers	OpenAI-compatible, DeepSeek, Anthropic, Ollama-style, local, or other configured model backends.	External dependency; trust depends on provider, deployment, data-processing terms, and secret handling.
MCP servers and tool providers	Optional tool servers reachable through the MCP registry and policy controls.	External or semi-trusted dependency; may perform actions or access data outside TokenStream.
Corpus source systems	URLs, uploaded files, object storage objects, repository snapshots, archives, and document sources.	Potentially untrusted input unless provenance and integrity are established.
Attacker	External or internal party attempting unauthorized access, data disclosure, tampering, denial of service, tool abuse, credential theft, or supply-chain compromise.	Untrusted.

6. Deployment Environment

The assessed deployment model is the repository's Docker Compose style topology. It is local-first and self-hosted. The deployment includes:

- dev-ui on the management/admin boundary;
- embedded config_auth for human authentication, RBAC, machine-key management, registry persistence, and runtime snapshot export;
- orchestrator-api as the normal client application boundary;
- retrieval-api as a trusted internal or deliberately exposed retrieval boundary;
- ingestion-worker as an internal background processing service;
- Qdrant for vector storage;
- MinIO-compatible object storage for uploaded source content;
- TEI embedder for embedding generation;
- SQLite-backed registry, lexical, and graph-aware retrieval data;
- runtime snapshot files for providers, policies, API keys, RAG settings, MCP servers, MCP settings, processors, and retrieval profiles;
- optional Open WebUI as a local chat UI client of the orchestration API.

Security posture depends heavily on operator-controlled deployment choices. For any deployment beyond a single trusted local machine, operators should define network segmentation, reverse proxy/TLS, exposed port policy, secret storage, backups, update monitoring, logging, resource limits, and incident response procedures.

7. External Systems And Services

External or separately managed systems include:

- hosted LLM providers and OpenAI-compatible model APIs;
- local or remote Ollama-style model backends;
- optional Anthropic or other provider integrations supported by provider adapters;
- MCP servers and the external systems those tools can reach;
- source systems such as URLs, repositories, document stores, uploaded files, and archives;
- Hugging Face model download endpoints when embedding models or gated models are used;
- container images and registries for TokenStream, Qdrant, MinIO, TEI embedder, Open WebUI, and base images;
- GitHub or other source hosting and release distribution channels;
- host operating system, Docker engine, Docker Desktop or container runtime, reverse proxy, TLS termination, DNS, firewall, and storage subsystem;
- optional monitoring, log aggregation, backup, and vulnerability scanning systems selected by the operator.

TokenStream should not assume that external providers, MCP servers, source systems, or host infrastructure are intrinsically trustworthy. Data sent across those boundaries should be limited, authenticated, authorized, and logged according to the deployment's risk profile.

8. Assets To Be Protected

Protected assets include:

- provider credentials and provider secret references;
- machine API keys, key hashes, key scopes, and key restrictions;
- human admin credentials, password hashes, session cookies, roles, permissions, and user records;
- internal service tokens, including config-auth internal token and orchestrator reload token;
- provider records, policy records, RAG settings, MCP settings, processor records, retrieval profiles, and runtime snapshots;
- corpora, source records, uploaded files, raw fetched source material, object storage objects, and source metadata;
- generated chunks, embeddings, Qdrant collections, SQLite lexical indexes, graph-aware tables, retrieval profiles, and retrieval results;
- prompts, model requests, model responses, tool requests, tool responses, structured outputs, and citation metadata;
- ingestion jobs, job plans, job statistics, errors, readiness state, and purge/delete state;
- audit events and security-relevant logs;
- Docker images, release artifacts, source code, dependency definitions, CI workflows, and documentation used to operate the product securely;
- availability of essential functions: authentication, admin access, model routing, policy enforcement, retrieval, ingestion recovery, configuration reload, and corpus readiness.

9. Sensitive Data

Sensitive data may include:

- provider API keys, machine keys, internal service tokens, admin passwords, session cookies, MinIO credentials, Open WebUI credentials, private keys, and any future vault-backed secrets;
- private or proprietary documents, customer support material, repository snapshots, compliance material, policy documents, and other uploaded or fetched corpus sources;
- extracted text, normalized blocks, chunks, embeddings, retrieval metadata, citations, and graph/lexical index records derived from sensitive source material;
- prompts, conversation context, tool arguments, tool outputs, LLM responses, provider error details, and structured response payloads;
- admin user records, RBAC state, API-key subject metadata, audit events, ingestion job metadata, and operational logs;
- corpus identifiers, tenant/environment IDs, source URLs, file paths, commit SHAs, line ranges, and metadata that may reveal sensitive deployment or business context.

Embeddings and indexes should be treated as derived sensitive data when generated from sensitive sources. They may not be plaintext copies, but they can still reveal or help infer protected content.

10. Interfaces

Interface	Typical service / path	Intended exposure
Orchestration API	`orchestrator-api`, `GET /v1/models`, `POST /v1/chat/completions`, `POST /v1/rag/query`, `POST /v1/rag/lookup`	Client application backend boundary.
Admin UI and management API	`dev-ui`, `/`, `/v1/auth/*`, `/v1/management/*`	Trusted human administrator and scoped service-account boundary.
Retrieval API	`retrieval-api`, `POST /v1/query`, `POST /v1/lookup`, `GET /corpora`	Trusted internal boundary unless deliberately exposed with controls.
Config/auth internal API	`dev-ui` / `config_auth`, `/internal/*`	Internal service-to-service boundary protected by internal token.
Ingestion worker APIs	`ingestion-worker`, health, dry-run, purge, job polling/claim/update paths	Internal operational boundary.
Runtime reload API	`orchestrator-api`, `/v1/internal/reload`	Internal control-plane boundary protected by reload token.
Qdrant	Compose port `6333`	Data-store boundary; should normally be internal.
MinIO API and console	Compose ports `9000` and `9001`	Object-store boundary; should normally be internal or admin-only.
TEI embedder	Compose port `8080`	Internal embedding service boundary.
Optional Open WebUI	Compose port `30000`	Optional local UI client boundary; not the TokenStream admin boundary.

Interface	Typical service / path	Intended exposure
Runtime snapshot files	`/runtime/*.json`	Internal configuration data shared between config/auth and runtime services.
Local persistent volumes	Qdrant, MinIO, config-auth DB, local `data` directory	Host/container storage boundary.
External provider APIs	HTTPS or local provider base URLs	External network boundary.
MCP server endpoints	Configured MCP server URLs	External or semi-trusted tool boundary.
Source URLs and upload endpoints	Management API source registration/upload	Input boundary for untrusted or semi-trusted content.

11. Trust Relationships

The following trust relationships are assumed for this assessment:

- client application backends are trusted only within the limits of their machine keys, scopes, policies, and network placement;
- client application end users are not trusted to access TokenStream directly;
- human administrators are trusted to manage security-relevant configuration, but admin sessions and credentials must still be protected;
- orchestrator-api trusts runtime snapshots exported by config_auth and must fail safely when provider, key, policy, RAG, or MCP configuration is missing or malformed;
- retrieval-api trusts registry data and internal calls from TokenStream services, but direct use by client applications requires a deliberate deployment decision;
- ingestion-worker trusts its job claims and registry definitions, but it processes potentially untrusted source content and should treat source bytes, URLs, archives, and metadata as hostile input;
- Qdrant, MinIO, SQLite stores, and local volumes are trusted only if deployment networking, storage permissions, and backups are controlled by the operator;
- LLM providers are trusted to process model requests according to their own service behavior and terms, but TokenStream should minimize unnecessary sensitive context sent to them;
- MCP servers are trusted only for explicitly allowed tools and should not receive broader credentials or data than required;
- the OSS maintainer is trusted for upstream source and best-effort vulnerability handling, but no contractual support, uptime, remediation deadline, or insurance obligation is assumed.

12. Trust Boundaries

Primary trust boundaries for threat identification are:

- between untrusted end users and the client application that fronts TokenStream;
- between the client application backend and orchestrator-api;
- between trusted administrators and dev-ui / /v1/management/*;
- between dev-ui / config_auth and runtime snapshot consumers;
- between internal services and config-auth /internal/* routes;
- between orchestrator-api and retrieval-api;
- between orchestrator-api and external or local LLM providers;
- between orchestrator-api and optional MCP servers;
- between ingestion-worker and external source systems, uploads, archives, object storage, and parser/processor logic;
- between application/runtime containers and data stores such as Qdrant, MinIO, SQLite, mounted volumes, and runtime JSON files;
- between the self-hosted deployment and upstream supply-chain sources such as container registries, package repositories, model downloads, and GitHub releases;
- between the operator's support expectations and the actual best-effort OSS maintenance model.

These boundaries should be represented in the Threat Dragon model and used to drive STRIDE threat enumeration.

13. Expected Period Of Use

The expected period of use is deployment-specific. TokenStream can be cloned, forked, run, and modified indefinitely by operators because it is open-source software. For this assessment, the assessed version is 0.1.0, and the expected use period should be treated as the period during which a specific operator continues to run that version or release line in a live environment.

Because there is no commercial support contract, operators must define their own operational use period, update cadence, vulnerability-monitoring practice, backup window, and retirement criteria. If a deployment remains active after upstream maintenance stops or after the operator stops applying updates, the residual risk increases and the deployment should be reassessed.

Minimum review expectation for this voluntary risk assessment:

- review at least annually while the assessed version or release line remains in use;
- review on every substantial architecture, exposed-interface, authentication, authorization, provider, MCP, ingestion, retrieval, dependency, or deployment change;
- review after any severe vulnerability, exploited dependency, secret exposure, incident, or support-period decision.

14. Support Period

There is no enterprise support period, no paid support commitment, no warranty, no service-level agreement, and no guaranteed vulnerability remediation timeline for the assessed OSS/non-commercial distribution.

For risk-assessment purposes, the support model is:

- vulnerability reports are directed privately to the project maintainer according to `SECURITY.md`;
- maintainer response and remediation are best-effort and depend on maintainer availability;
- release and patch availability is not guaranteed by contract;
- operators remain responsible for monitoring dependencies and releases, applying updates, rotating secrets, hardening deployments, and deciding whether to fork, patch, migrate, or retire an instance;
- deployments that require assured support, incident response, or regulated product obligations need their own accountable maintainer, support process, security contact, and support-period policy;
- no CRA CE marking, EU Declaration of Conformity, or market-placement claim should be inferred from this voluntary risk-assessment documentation.

If a future maintainer or distributor offers a formal support period, this section must be replaced with the actual support-period start, end, scope, security update commitments, vulnerability disclosure route, and end-of-support handling.

15. Evidence Sources Used

- `README.md`
- `overview/what-is-tokenstream.mdx`
- `overview/behind-your-application.mdx`
- `concepts/core-concepts.mdx`
- `concepts/policies-and-model-access.mdx`
- `concepts/retrieval-and-source-backed-context.mdx`
- `guides/connect-a-client-application.mdx`
- `guides/operate-tokenstream.mdx`
- `quickstart/configure-access.mdx`
- `reference/api-reference.mdx`
- `reference/configuration-reference.mdx`
- `docs/services_overview.md`
- `docs/service_orchestrator_api.md`
- `docs/service_dev_ui.md`
- `docker-compose.yaml`
- `SECURITY.md`

- `packages/config_auth/app/main.py`
- `packages/config_auth/app/db.py`
- `packages/config_auth/app/security.py`
- `services/orchestrator_api/app/auth.py`
- `services/common/common/auth.py`